

3.Agents底层逻辑之前向传播 - 图文解说

01. 回顾单个神经元



场景 1

完整内容

在我们这个非常简洁的神经网络里，有多少参数呢？有两万六千多个参数，不多，不多。我们平时看到的一些大模型有多少呢？像 DeepSeek，是有一点五万亿个参数。现在知道参数是什么意思了吧？

我们先回顾一下上节课的内容。上节课讲了单个神经元：接收一个向量的输入，产生一个标量的输出。当输入的向量固定时，它的输出结果唯一取决于它的参数，参数就是权重加上偏置。我们希望输出结果符合预期，就要不断调整这些权重和偏置，我们当时就是这样说的：希望把它们调整到一个合适的位置，使它在这个输入的情况下，能够达到预期的输出。这就是上节课的内容。

但是，实际的神经网络可不止一个神经元，它是由很多很多神经元连接起来的。

画面说明

浏览器显示单个神经元的演示：左侧三个输入值通过权重连到中间节点，右侧给出输出值；控制区可调整权重与偏置。讲解开头与参数统计段出现的“DeepSeek一点五万亿”是课程原述，未做型号及规格核验。

02. 网络分层与层编号



场景 2

完整内容

一个神经元的结构太简单了，也不符合人类大脑有很多神经元的结构。那么这些神经元怎么连接呢？这就是这节课要学习的内容，叫前向传播。

画面中的每一个小圆圈都是一个神经元。这套神经元是分层次的，一层一层：第一层、第二层、第三层、第四层。这些层还有特殊的名字：第一层叫输入层，中间这两层叫隐藏层，最后一层叫输出层。

输入层和输出层很好理解。输入是一个向量；这里每个节点可以理解为会输出一个数字，这一组数字就是输入，之后会具体讲。输出层就是我们的结果。你可以把整个过程看成一个函数：输入一个东西，实际上是向量，经过一系列运算，最后产生一个结果。输入一个东西，经过这些运算，产生最后的结果。中间的隐藏层就是一些中间运算过程。

整个神经网络，你就把它想象成一个函数：输入，算，算完了输出。这是它的整体结构，我们先有一个整体认识。输入层也叫第零层，然后从隐藏层开始，第一层、第二层、第三层往后数。这也是一种叫法。可以叫输入层、隐藏层、输出层，也可以叫第零层、第一层、第二层、第三层。

画面说明

前向传播界面把节点排成四列：输入层、隐藏层1、隐藏层2、输出层。粉色标注框和箭头强调各列及从左到右的方向。

03. 隐藏层的规模与识别任务



场景 3

完整内容

输入层和输出层都是固定只有一层。中间隐藏层有多少层呢？不一定。不同模型、不同神经网络架构，隐藏层是不一样的。比方说 DeepSeek V4 这个版本，好像有六十多层隐藏层；GPT 就不知道了，因为它是闭源模型，你不知道它有多少隐藏层。业界猜测它应该有一百多个隐藏层。

但隐藏层不是越多越好。设置多少合适？不一定，要看实际工程经验。没有一个严格的数学论证说一定设置多少层才合适。太多容易产生问题，太少也容易产生问题，两种情况都会容易产生问题，设置一个合适的值最好。不同模型架构的设置不一样，这还跟训练的数据集有关系。

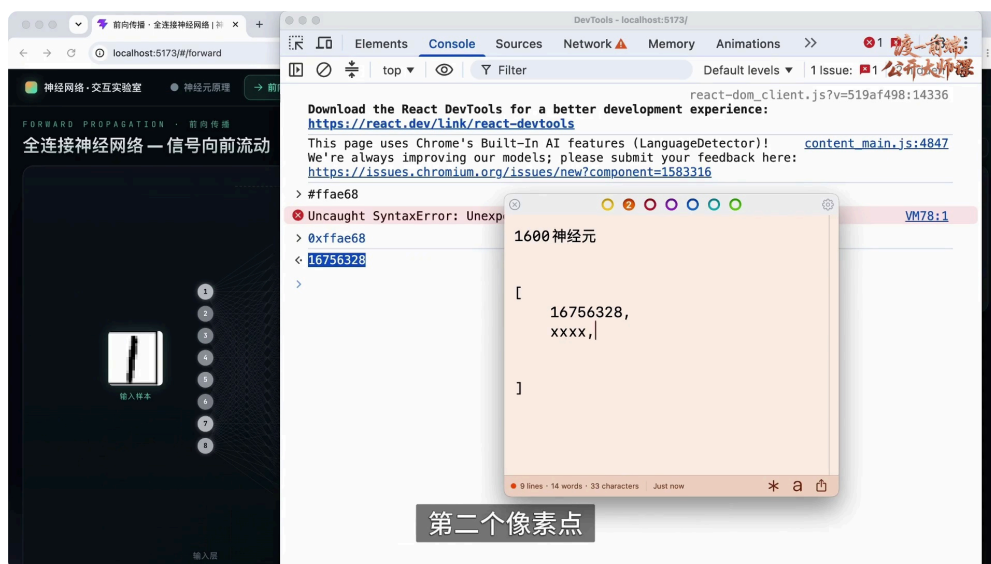
然后每一层有多少个神经元呢？像这个隐藏层，每一层有多少个神经元？你看，一个个节点都有编号，但数量也是不确定的。像 DeepSeek，每个隐藏层有七千多个神经元。有些架构里，不同隐藏层的神经元数量还不一致。GPT 有多少个，我就不清楚，因为它是闭源模型；开源模型才能知道这些信息。

接下来我们研究这么多神经元是怎么连接、怎么运算的。假设整个神经网络要做一件事，从头到尾怎么传递、怎么输入、怎么输出？理解这个过程，这节课就结束了。先别着急点“下一步”。我们举个例子：希望这个神经网络产生一种智能，能够识别一张小图片。

画面说明

演示保持四列网络结构，输入区显示手写数字样本，右侧有“下一步”按钮。示意图只画出少量输入节点，不能据此把示意数量当作真实像素数量。本段关于DeepSeek版本、隐藏层数量、每层神经元数及GPT层数猜测，均按原讲解保留；不是本教程核验后的模型规格。

04. 把图片变成输入向量



场景 4

完整内容

这里有很多随机小图片，比如手写的数字2、3、看起来像1的数字，还有4。我们希望它能识别这些数字。输入是什么？自然是图片本身。但它怎么输入，神经网络能接收图片吗？不能，整套神经网络只能接收数字，它不能接收别的东西，只能接收数字。所以我们肯定要把图片数字化。

数字化的手段很多，不同神经网络架构可能有不同方式。我们用最简单的方式：假设图片是40×40，宽高都是40，一共多少像素点？1600个。所以输入层就设计成1600个节点。这里实在画不下1600个，我就画了8个。若这个网络基于40×40的图片，按照这里的表示方式，就设置1600个输入节点，明白这个意思。

每个节点要接收输入，给它什么呢？一个像素点对应一个数字。平时设置颜色是 RGB，三个数字；三个数字是不是可以合并成一个？比如十六进制颜色值，这个十六进制值，不就是一个数字吗？于是可以把这个值作为第一个数字，表示第一个像素点。再接第二个像素点、第三个像素点，是不是一共写1600个像素点？是不是可以这样？把这组数字是不是就可以作为输入？这组数字就可以作为输入。

输入以后，输入层这些节点就点亮了。点亮是什么意思？就是激活值。那个值越高，它就越亮，就是这个意思。输入层比较特殊：前面讲的神经元有权重、偏置，但输入层没有这些。你可以把它想象成专门提供用户输入信息的地方，也就是要训练的数据的输入信息。

假设第一个节点比较亮，可能对应一个比较大的数字；第二个暗一些，对应另一个数；第三个介于两者之间。总之，把要训练的图片数字化后，输入层显示的就是数字化的结果。输入层里不做转换，它表示原始输入。每个节点要往后传递一个值，输出一个值。这里向后面传递的结果，就是我们的输入，这一块能理解吗？如果训练这张图片，就有1600个值、1600个结果往后输出。

输出以后谁来用这些结果？接下来就到第一个隐藏层。

画面说明

界面中的手写数字小图对应输入列。便签展示“1600神经元”和一组数字，浏览器控制台出现颜色十六进制示例。节点明暗用来表示数值大小。

05. 隐藏层配置与激活亮度



场景 5

完整内容

隐藏层设置多少个神经元？这里你可以认为我是随便设置的，怎么好看怎么画。实际工程落地时，有多少隐藏层、每层多少神经元，要看经验。你们不参与模型训练，这个配置不需要自己管，相关人员会设置好。甚至在训练过程中发现不太对劲，还可能加一层、减一层，加一些神经元或减一些神经元，根据工程经验慢慢调整。

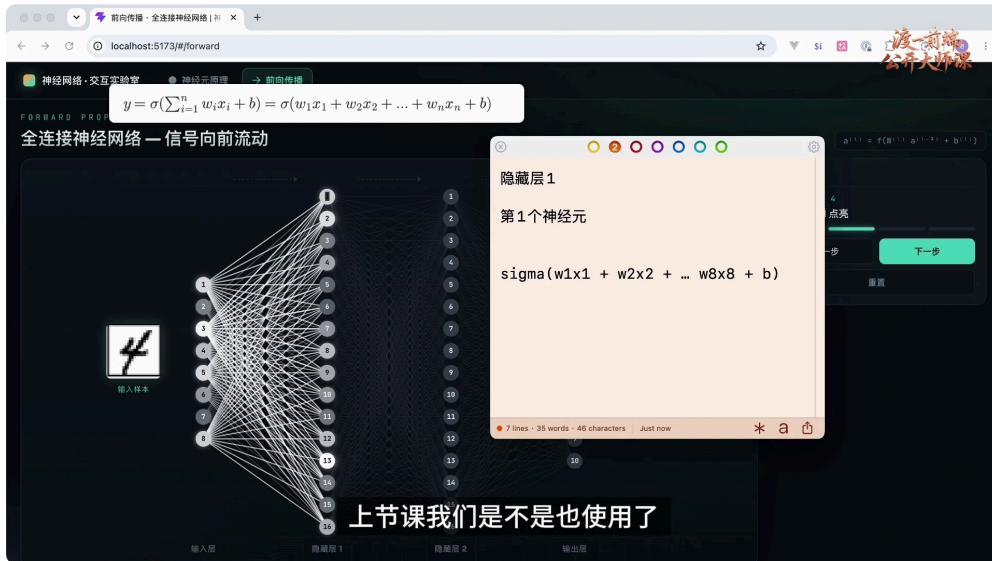
那么每个神经元具体在干什么、负责什么？没有人清楚。最多能管有多少层、每层有多少神经元；这个神经元负责什么、另一个神经元负责什么，都不知道，谁都不知道。最多最多只能管有多少层、每层有多少神经元，具体每个神经元负责什么，不是我们人为规定的，也规定不了。智能是它自己产生的。

现在有输入了，每个输入节点产生一个数字，整体就是一串数字。点下一步，到达第一个隐藏层。图像上做了直观展示：越亮的神经元，往后传递的数字越大；越暗的神经元，越不激活、不活跃，往后传递的数字就越小。我们可以认为数字小表示不活跃，数字大表示活跃。数字很小，就说明它不活跃；数字越大，就说明它很活跃，往后传递的数字很大。这里所谓活跃和不活跃，其实就是数字大小，只是这样说让你听起来更直观一些。

画面说明

第一隐藏层中的16个节点具有不同亮度，输入列到隐藏层的线条可见。明暗是演示中的数值编码，不表示人为指定的识别职责。

06. 第一个隐藏神经元的计算



场景 6

完整内容

下一层有很多神经元，每一个的运作方式都跟上节课讲的一模一样。把上节课的公式拿出来。我们看第一隐藏层的第一个神经元，它怎么算？这里没有神奇、没有魔法，全是数学运算。就像让一个小学生做数学题，花的时间可能非常长，但慢慢算也能算完，慢慢算就是了。

第一个神经元的输出先要用权重。权重从哪里来？全随机。随机也有方法，这里说是通过正态分布来随机，你暂时不用管，先理解为随机的一个数字 w_1 。随机出来一个 w_1 ，随机出来的数字乘以 x_1 。那么 x_1 从哪里来呢？来自前一层第一个节点的输出。再加上随机的 w_2 乘以上一层第二个节点的输出 x_2 。

你看，这里是全连接。我们这节课学习的是全连接神经网络，神经网络也有非全连接的。现在很多 GPT 是非全连接的；先理解全连接，其他非全连接可以先理解为少算一些。我们现在讨论的是第一个神经元的计算。数学基础稍弱的同学可能要多次暂停，不断去想，不断去想，不断去消化，才能理解这一课。

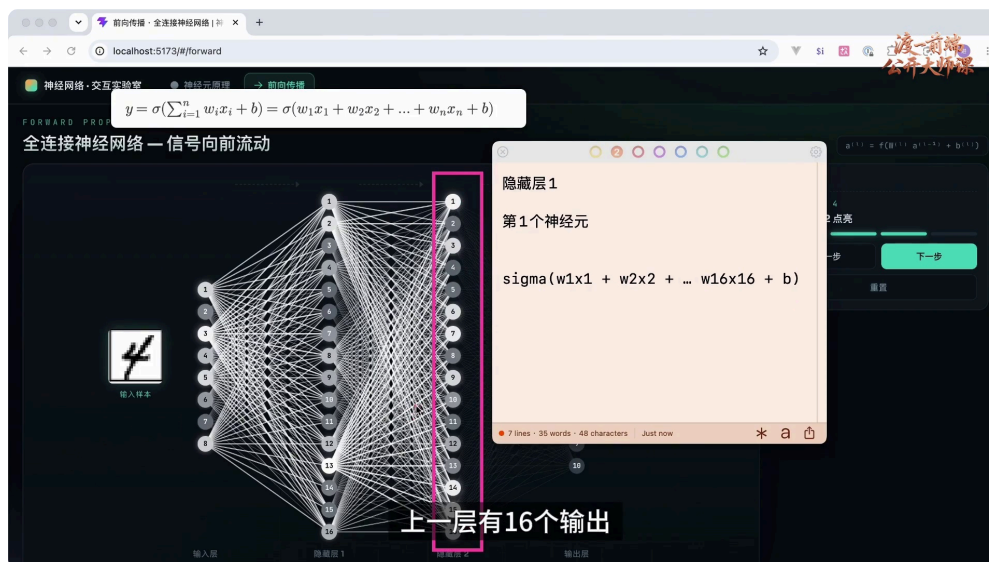
我重置一下，你看这个节点是不是跟前面所有节点都有连接？每一个神经元跟前面所有神经元都有连接，看到了吗？都是这样的，所以叫全连接。你看，每一个神经元，跟前面全部都是有连接的。到这一层时， w_2 乘 x_2 ， x_2 来自上一层输出：把上一层的输出作为这一层的输入。然后一直加到 $w_8 x_8$ 。实际上这里不是 8，而是 1600，因为我们假设图片为 40×40 、输入层有 1600 个值，就要进行这么多项运算。我这里写 8 只是示意。

一直加下去，最后再加上一个 b ， b 也是随机的。权重都是随机的， x 都来自上一层的输入，偏置也是随机的。外面还有一个固定的激活函数，这里说“叫 Sigmoid 函数，Sigmoid 函数的实现有很多种”，并提到上节课用过“除最大值”的方式。函数固定下来，就可以算出结果。把这些数带进公式，做这道数学题就可以了。

画面说明

演示突出第一个隐藏节点与全部输入节点之间的连接。上方公式为 $y = \sigma(\sum w_i x_i + b)$ ，便签逐项写出 $w_1 x_1$ 、 $w_2 x_2$ 直至末项，再加 b 。阅读提示：本段原述把 Sigmoid 与“除最大值”混用，也把非全连接概括为“少算一些”；这里保留教学原意和说法，不把这些简化措辞当成严密定义。

07. 同层参数与逐层计算



场景 7

完整内容

现在这些像素颜色值都拿到了，颜色信息很好获取，你就挨个算。花五六天也许能算完第一个神经元，然后接着算第二个。这是说你确实能手工算，只是慢。这样是不是就可以算出这一层所有数值、得到所有结果？

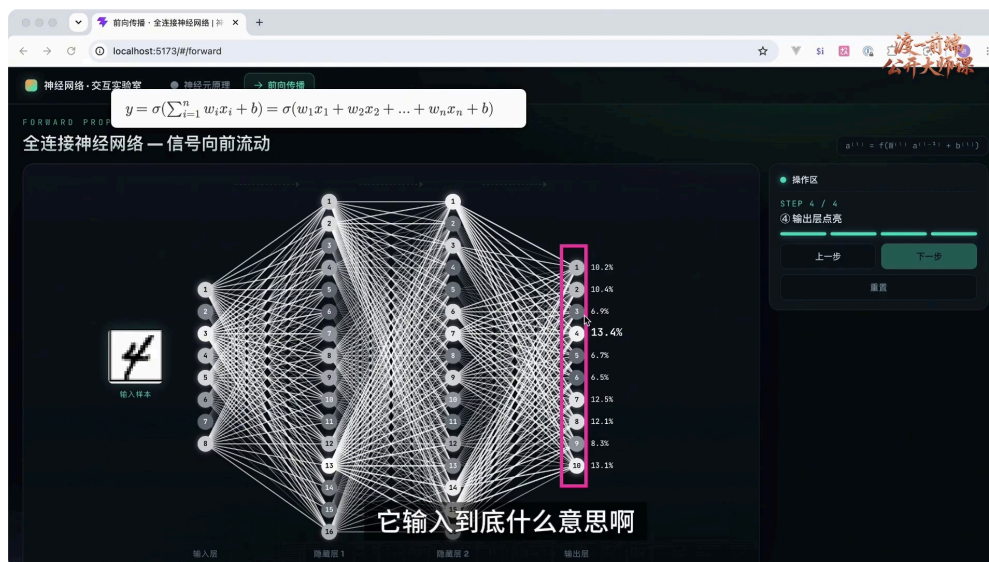
好，又来这一层，每一个都是随机的。第二个神经元同样的算法，第三个神经元同样的算法。每个神经元有自己的权重和偏置，权重不一样、偏置不一样，但这里使用的函数是一致的。输入都来自上一层。也就是说，对于这一层的每个神经元，接收到的 x_1 、 x_2 直到 x_8 完全相同；因为各自参数不同，算出的结果也不一样。所以每一个算出的结果是不一样的。这样是不是就算出了这一层所有神经元？是不是全部算出来了？

接下来是不是又到下一层，一样的过程？可以暂停想一想。下一层又来，有它自己的 w_1 ，有自己的权重，又是随机的。然后是 w_2 ，一直加到 $w_{16 \times 16}$ ，因为上一层有16个输出。每个神经元有多少个权重，等于上一层有多少个输出。上一层有16个输出，这一层每个神经元就得有16个权重；上一层有8个输出，这一层每个就得有8个权重。每一层的神经元有多少个权重，等于什么？就等于上一层有多少个输出。这里可以再暂停想一想。

画面说明

第二个隐藏层接收上一层16个节点的输出，网络连线逐层展开。便签把求和末项从输入示意中的第8项改为第16项。

08. 输出计算与结果含义



场景 8

完整内容

这样又算出一层结果，再到下一层，重来一次，基于上一层算出结果。最后到输出层，它也是一样的，没有特别之处，还是权重乘以输入，再完成后续运算。输入就是上一层输出，上一层的输出就是下一层输入。这里每个输出节点也是乘到16项。

把同样的计算重复多次：第一次、第二次、第三次、第四次、第五次、第六次、第七次、第八次，就可以把这一层的输出算出来。

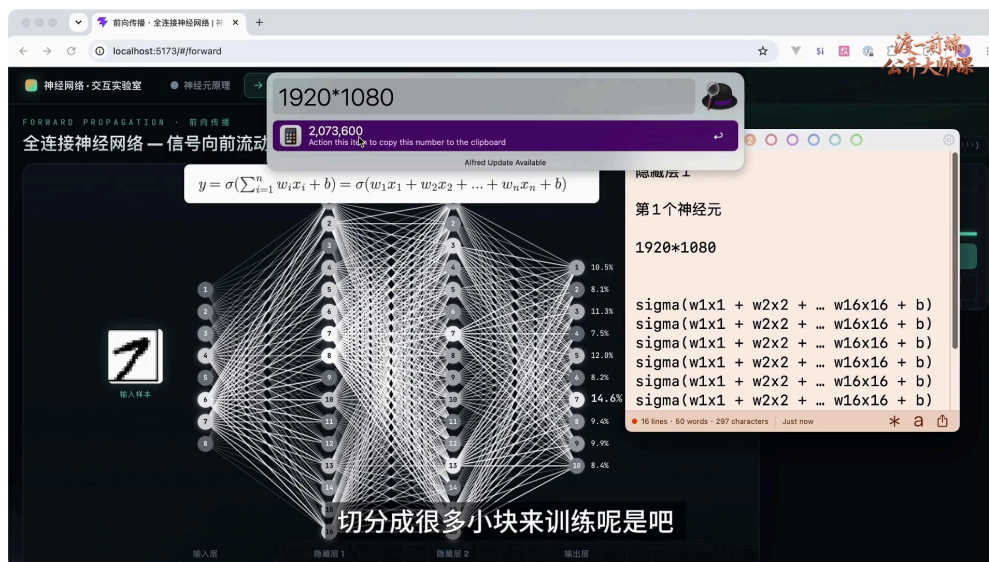
算出来之后，数字有大有小。接下来要给输出层定义意义。上节课说过：单个神经元输入的东西到底是什么意思，输出的东西又是什么意思，它自己不知道。反正就是接收数字，运算后输出数字。它自己有智能、有像人一样的意识吗？你找一找，意识长在哪里？它就是接收数字，计算过后输出。它自己有智能吗？像我们人一样有意识吗？你说意识从哪儿来？这里就是输入、输出、输入、输出。输出到底是什么意思，是我们人为规定的。

到了最后一层，我们怎么定义结果？这里给输出层安排了10个神经元。为什么？输出层和输入层的数量都要设计。刚才输入层是按训练图片有多少像素点，设置多少输入。如果有些图片尺寸不固定，还想放在一起训练，怎么办？就要做归一化等处理。

画面说明

网络已经连接到输出列，输出列在画面上实际列出10个节点和对应百分比。应以这个可见数量理解本例，不能把讲解中一段“八次”的口头示意当作本例最终输出数。

09. 输入尺寸和预处理



场景 9

完整内容

比如假设这个神经网络最大接收图片的分辨率是 1920×1080 ，输入层应该有多少节点？按这里一个像素一个数的讲法，就是约207万个。这个数量确实大。所以现在处理图片时会做一些处理，让它变小。

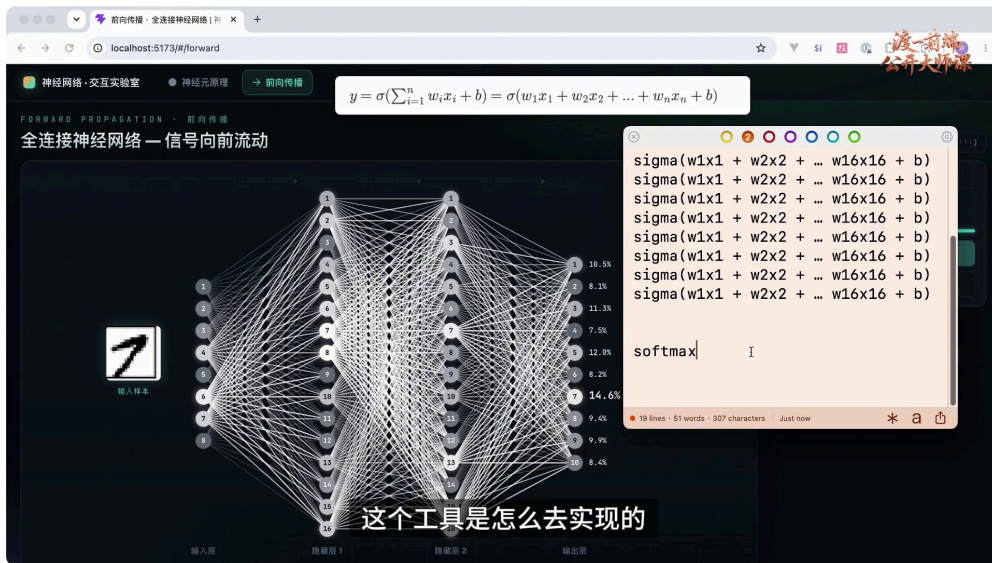
举个题外话：训练图片时，为什么一定要整张图片拿来训练？可以把它切成很多小块来训练，输入节点数量就能缩小一点，为什么一定整张图一起输入呢？如果一定整体输入，按照刚才的表示方式，就需要约207万个输入，实在太大了。

如果图片没有达到这个分辨率怎么办？可以拉伸到这个分辨率；也可以不拉伸，只点亮部分输入节点，剩下的不点亮。如果超过这个分辨率，做什么呢？做裁剪、压缩，把它压到这个分辨率。这些都是题外话。这些属于题外话。总之，我们这里讨论的神经网络输入层，基本上是固定的；有些特殊情况下不固定，但不在本课讨论范围。

画面说明

便签出现 1920×1080 ，浏览器仍显示示例网络。讲解讨论的是固定输入大小、切块以及拉伸、补空白或裁剪压缩等预处理思路。

10. 输出定义、Softmax与随机结果



场景 10

完整内容

回到刚才的话题：输入层的神经元数量要设计，输出层也要设计，因为输出是结果，要赋予它意义。输出层按部就班运算，结果有大有小。我们就可以定义：某个输出越大，就认为对应数字的可能性越高。比如7对应的结果最大，就认为识别成7的概率最高；4很暗，输出数字不大，就认为是4的概率小。

我就给它人为定义，纯粹是人为定义。这些东西都是人为定义的，人为给它定义意义。给它一张数字7的图片，我就期望它输出7的概率最大。但它本来输出的是概率吗？不一定，可能是几千、几万这样的数，你不知道是什么数字。一般会使用数学工具，比如著名的 Softmax 函数，它可以把任何一组数据都转换成百分比形式。有兴趣可以了解这个工具怎么实现。不管那组数据有多大，它都可以转换成百分比格式。

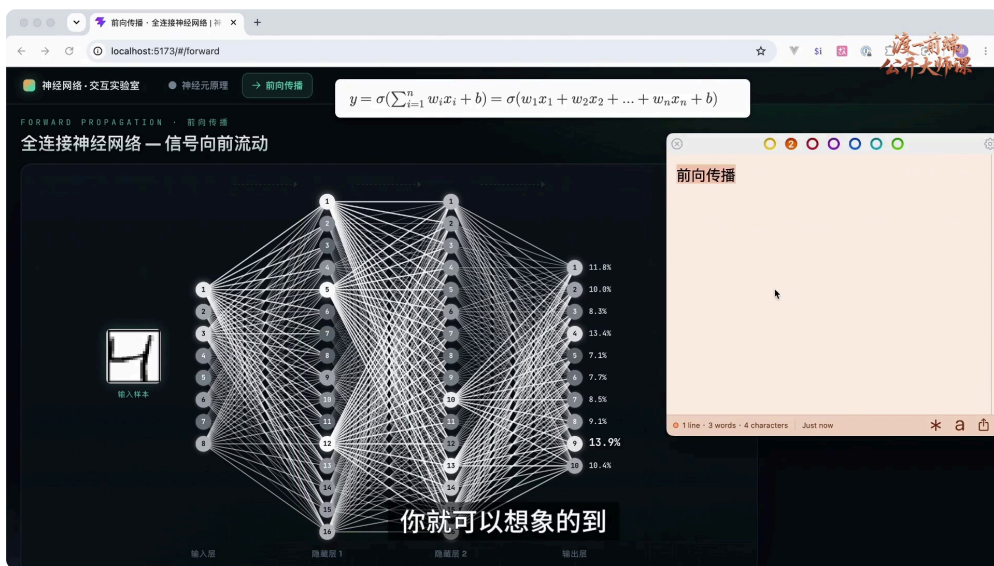
这里的演示也用了这个工具。实际输出并不是屏幕所显示的百分比，而是转成百分比后再展示。明白我的意思吗？你就可以这样想象：假设某个值是0.146，显示成百分比就是14.6%。图中7的可能性最高，当然这里只是凑巧。

这里权重和偏置全是随机的，只有简单数学运算，怎么可能稳定识别准确？它不知道自己在干什么，只管算，算出来什么就是什么，结果是散乱、没有意义的。算错了，我们要告诉它错了，但怎么告诉它不是这节课讨论的问题。本课只讨论所有神经元从输入层传到输出层的完整过程。可以暂停，再回忆一下它是怎么做的。

画面说明

输出节点旁显示一组百分比，示例中7对应14.6%，数字值最大的节点较亮。图示只是随机参数下一次前向计算的结果，不是训练成功的证据。

11. 前向传播与参数的共同作用



场景 11

完整内容

这个完整过程有个专用名词，叫前向传播。本课讲的就是前向传播。它是训练神经网络的第一个阶段：先给一条数据，让网络跑一次，得到结果；刚开始的结果肯定乱七八糟，后面再调整。这就是你第一个要认识的名词。先让它跑一次，给它一个数据，让它跑一次，后面我们再调。

第二点，从输入到最终输出，网络并不能控制你给它什么输入，输入是我们的数据。听好这句话，当输入是固定的时候，听好这句话：当输入固定时，是什么东西影响输出结果？想一想上一课：单个神经元输入固定时，影响输出的是不是权重加偏置？

换到整个神经网络也一样。除了输入层，隐藏层和输出层的每个神经元都有自己的权重与偏置。输入层没有这种神经元，第零层不带神经元。除了输入层之外，其他的隐藏层包含输出层，每个神经元是不是都有自己的权重、自己的偏置？整个网络的全部权重、偏置集合，共同影响最终结果。

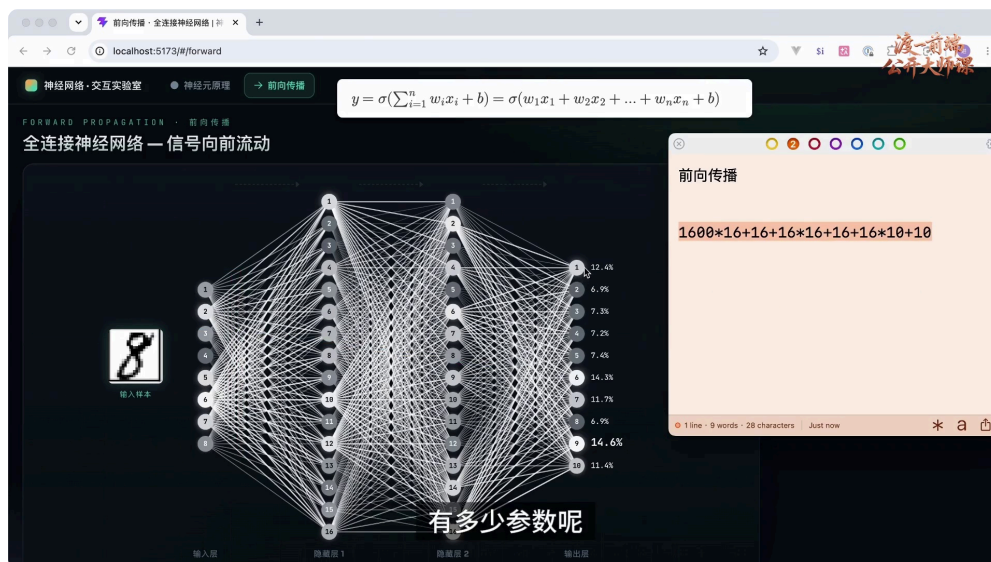
要调整这个结果，你可以想象有非常非常多的旋钮。调整每个旋钮，都会对网络输出造成影响。调整每一个旋钮，都对整个神经网络的输出结果造成一定影响。这个是人来调吗？肯定不是人来调，它会自己调。怎么自己调，下节课再说。

当输入固定时，权重和偏置决定输出。因为一个神经元的亮度、也就是它输出的数值，会影响后面的神经元；后面的节点又有自己的权重和偏置，影响自己的输出，再影响更后面的节点。然后它的输出结果又会影响后面，这种影响是传递性的，最终整个网络的权重、偏置都会影响输出。

画面说明

画面显示整个网络的隐藏层与输出层，旁边便签标题为“前向传播”。各层明暗和连接展示了一个节点的输出如何传向后续层。阅读提示：原讲解前文把输入节点也称为神经元，此处又说输入层“没有神经元”；结合上下文，其区分重点是输入节点不带可训练权重、偏置，术语本身存在简化。

12. 逐层计算参数数量



场景 12

完整内容

最终整个网络的权重和偏置都会影响输出。大家可以算一下：这个简单网络有多少参数？假设图片为 40×40 ，输入层有1600个输入。输入层有没有权重、偏置？没有，它就是提供输入的。我们要算权重、偏置，就从这一层开始算。它有多少个权重、偏置，能反应过来吗？先看第一个神经元。

先看第一个神经元有多少权重。每个神经元的权重数量，等于上一层传来多少个输入。每传来一个 x ，就要配一个权重。上一层是1600个输入，所以这一层每个神经元有1600个权重。一共有16个神经元，这一层的权重数量就是 1600×16 。偏置呢？一个神经元一个，所以还有16个偏置。

下一隐藏层一样：上一层有16个输出，所以每个神经元有16个权重。这一层有16个神经元，权重就是 16×16 。偏置 b 有几个？也是16个，对不对？所以再加16。

输出层呢？上一层有16个输出，每个输出神经元有16个权重；输出层有10个神经元，所以权重是 16×10 ，再加上输出层的偏置，10个。这就是整个网络。输入列只是画了8个，实际上按本例是假设1600个。

把它们合起来，这个非常简洁的神经网络里，就有两万六千多个参数。一个非常简洁的网络，也有两万六千多个参数，不多，不多。我们平时看到的模型，像 DeepSeek，讲解中举的是一点五万亿个参数。现在知道“参数”是什么意思了吧？就是权重加偏置，整个神经网络的权重与偏置。

最后看一些数学公式，作为了解就行。讲这些主要是为了引出一个点，其他不是最重要的。即使不看公式，前面的图也能帮你理解。首先是单个神经元的公式，上节课已经见过，这里不再重复解释。

画面说明

便签逐层列出权重与偏置计算： $1600 \times 16 + 16$ 、 $16 \times 16 + 16$ 、 $16 \times 10 + 10$ 。随后切换到课件文档，展示单个神经元求和加偏置并激活的公式。依画面算式复算的精确合计为26058（这是算式核对值）。

13. 从单神经元到层编号公式

完整内容

在神经网络里，单个神经元的公式写法可能会变一下。以后看模型训练的视频或文章，可能见到这种形式。道理一样，只是 x_i 写成了 a_i ，换了个名字。 x 往往表示最初用户输入的东西，但一个隐藏神经元的直接输入，不一定是最开始的数据，而是上一层的输出，所以换一个符号。

上面的 $L-1$ 不是指数，不是在说平方或立方，它表示的是层，来自第几层。不是平方，也不是立方，不是这个意思。比如当前是第 L 层的第 j 个神经元：输入层是第零层，第二层第一个神经元，可以写为 $a_1^{(2)}$ ；第二层第七个神经元，就是 $a_7^{(2)}$ ，意思是第二层的第七个神经元。下标是这一层的第几个，上标表示层。

那么它的输出等于什么？等于对上一层的每个输入做原来那套计算。 $L-1$ 就是上一层，把 x_i 换成 $a_i^{(L-1)}$ ，其他没有变化。只是把 x_i 变成 a_i ，上面写 $L-1$ ，来自上一层的输入。其他没有任何变化，还是单个神经元。

但真正运算时，是一个神经元算完再算下一个吗？不是，可以一起算。为什么？因为可以把一整层看成一个整体，写成矩阵。

画面说明

课件展示 $a_j^L = \sigma(\sum w_{ij} a_i^{L-1} + b_j)$ ，下方标明 L 是第 L 层， j 是当前层第 j 个神经元。随后开始展示整层矩阵形式。

14. 矩阵计算与GPU

神经网络中的单个神经元

$$a_j^L = \sigma\left(\sum_{i=1}^n w_{ji} a_i^{L-1} + b_j\right) \quad (2)$$

L : 第 L 层
 j : 当前层第 j 行神经元

神经网络中，某层的计算过程

完整矩阵形式:

$$\begin{bmatrix} a_1^{(l)} \\ a_2^{(l)} \\ \vdots \\ a_n^{(l)} \end{bmatrix} = \sigma \left(\begin{bmatrix} w_{11}^{(l)} & w_{12}^{(l)} & \cdots & w_{1m}^{(l)} \\ w_{21}^{(l)} & w_{22}^{(l)} & \cdots & w_{2m}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n1}^{(l)} & w_{n2}^{(l)} & \cdots & w_{nm}^{(l)} \end{bmatrix} \begin{bmatrix} a_1^{(l-1)} \\ a_2^{(l-1)} \\ \vdots \\ a_m^{(l-1)} \end{bmatrix} + \begin{bmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_n^{(l)} \end{bmatrix} \right)$$

简写形式:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) \quad (3)$$

每一个神经元的输出

$\mathbf{a}^{(l)}$: 第 l 层的输出向量 (大小为 $n \times 1$)
 $\mathbf{a}^{(l-1)}$: 第 $l-1$ 层的输出向量 (大小为 $m \times 1$)

场景 14

完整内容

把第 L 层第一个神经元的所有权重排一行，第二个神经元的权重排下一行，第三个、第四个，一直到第 n 个神经元，就得到一个矩阵。数学基础好的同学看一看能理解，我也稍微解释一下；数学基础不好的同学不用纠结，因为我讲这个的主要目的不是讲数学公式，我马上要引出一个东西。

这个权重矩阵乘以上一层的输入。你看，输入标着 $L-1$ ，确实来自上一层。它们乘出来——这里讲解称为“点乘”——乘出来是什么呢？你先别管，它可以乘。然后再加这一层的偏置。每层既有权重，也有偏置；加出来的结果再经过 σ 激活函数运算，得到的结果仍然按一列排列。

算出来的结果就是这一列，是一个矩阵。这个结果表示当前层每个神经元的输出：第一个的输出、第二个的输出，一直到最后一个。为什么讲矩阵？因为矩阵运算的性能很高，可以使用 GPU 加速。这就是现在训练模型、运行模型常使用 GPU 的原因：里面有大量矩阵运算。矩阵运算交给 CPU 和交给 GPU，讲解强调后者非常快，因为它可以批量运算。这是为什么要用 GPU 来计算的原因，就在这里。我就想跟你说这一点。其他公式能看懂就看懂，暂时看不懂也没关系。

矩阵还能简写：权重矩阵用 $\mathbf{W}^{(L)}$ 表示，上一层输入用 $\mathbf{a}^{(L-1)}$ 表示，这一层偏置用 $\mathbf{b}^{(L)}$ 表示，于是整层可以写成 $\mathbf{a}^{(L)} = \sigma(\mathbf{W}^{(L)} \mathbf{a}^{(L-1)} + \mathbf{b}^{(L)})$ 。理解就理解，不必强求。最后还有整个神经网络的参数量公式，前面已经算过。

画面说明

课件给出权重矩阵、上一层输出列向量、偏置列向量，以及整层输出列向量。红框按行突出不同神经元的权重；下面有 $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$ 的简写。阅读提示：正文保留原述“点乘”，图中实际展示的是矩阵与列向量相乘；可按权重矩阵各行分别与输入做内积来理解。CPU比较语句的原始ASR有不清晰词语，正文仅保留其明确的GPU批量加速含义，未声称完成听音核查。

15. 参数公式与训练预告

大纲

课件.md - 已编辑 353 词

$[a_n^{(l)}] \quad [w_{n1}^{(l)} \quad w_{n2}^{(l)} \quad \dots \quad w_{nm}^{(l)}] [a_m^{(l-1)}] [b_n^{(l)}]$

简写形式:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

其中:

- $\mathbf{a}^{(l)}$: 第 l 层的输出向量 (大小为 $n \times 1$)
- $\mathbf{a}^{(l-1)}$: 第 $l-1$ 层的输出向量 (大小为 $m \times 1$)
- $\mathbf{W}^{(l)}$: 第 l 层的权重矩阵 (大小为 $n \times m$)
- $\mathbf{b}^{(l)}$: 第 l 层的偏置向量 (大小为 $n \times 1$)
- $\sigma(\cdot)$: 激活函数 (如 Sigmoid、ReLU 等)

公式

$$\begin{aligned} \text{总参数量} &= \sum_{l=1}^L (n_l \times n_{l-1} + n_l) \\ &= (n_1 \times n_0 + n_1) + (n_2 \times n_1 + n_2) + \dots + (n_L \times n_{L-1} + n_L) \end{aligned} \quad (4)$$

你自己可以对照一下图啊

场景 15

完整内容

整个网络的参数量公式，可以自己对照前面的图想一想。公式知道也好，不知道也好，不影响后面学习。

这就是这节神经网络课程，叫前向传播。前向传播会得到一个结果，尤其没有训练好的网络，结果肯定乱成一锅粥。所谓训练，就是不断调整神经网络的参数，把它们调到合适的值，让输出有意义、符合预期。训练就是调整参数，调整权重，调整偏置，调的就是这个东西。怎么调，是下节课的内容。

这一课先要理解前向传播。课后可以在脑袋里再把整个过程过一遍。如果这一节理解得很模糊，下节课会很有难度。

画面说明

课件末尾展示总参数公式和逐层展开形式，讲解以训练就是调整权重、偏置收束。画面总参数公式： $\sum_{l=1}^L (n_l \times n_{l-1} + n_l)$ ，即各层权重数量加偏置数量。